

A New Serious Game for INSA Business Classes: A Web-Based Rugby Team Management Simulation

Abdessalam AITHAQI, Armand GHIONDA, George PAPAILIOPOULOS, Daniel DE ALMEIDA, Domagoj JENJIC

*Département Génie Électrique et Informatique, 4ème année
Institut National des Sciences Appliquées de Toulouse, France*

Abstract - Serious games have become a recognised pedagogical instrument in higher education, particularly in management curricula where they support active, experiential learning. At INSA Toulouse, the third-year business course currently relies on a proprietary simulation called SIMGEST, whose limited accessibility, awkward onboarding and inability to scale to large cohorts increasingly hinder its use. This article presents the design and first implementation of a web-based serious game intended to replace SIMGEST while providing future student teams with a clean, extensible foundation. The game is built around the management of a rugby team competing in a six-round tournament against three other teams, where players make recruitment and marketing decisions whose outcomes are mediated by player statistics, fan dynamics and a controlled luck factor. A central design goal is the strict separation between a generic "platform engine" (authentication, round orchestration, persistence, role-based access) and the "game rules" (rugby-specific entities and formulas), so that future cohorts can introduce new variables and mechanics without rewriting core components. The system follows a decoupled architecture: a TypeScript/Express REST API backed by a relational schema managed through the Prisma ORM, and a React/Tailwind single-page frontend. A first complete loop (login, team assignment, decision submission, simulation, results) has been implemented and tested. Initial play-tests indicate that the gameplay balance is plausible across the six rounds and that the platform behaves predictably when used by four parallel teams. Beyond its immediate pedagogical use, this work illustrates how a small student team, working under realistic institutional constraints, can deliver a maintainable serious-game foundation that future teams can extend toward a fully featured business simulator.

Keywords - Serious games; gamification; business simulation; web application; rugby management; educational software; role-based access; agile development; scalable architecture.

I. Introduction

Over the last two decades, serious games, that is, digital games designed for a primary purpose other than entertainment, have moved from a marginal curiosity to a recognised teaching tool, particularly in domains where decision-making, resource allocation and risk management can be safely rehearsed in a simulated environment [1], [2]. In management education, business simulation games (BSGs) have become a common complement to lectures and case studies: rather than reading about a firm, students operate a simulated company, observe the consequences of their decisions over several rounds and progressively refine their strategy [3], [8]. Multiple systematic reviews and meta-analyses now support the claim that, when properly integrated into a course, such games can

produce learning outcomes that are at least comparable with, and often superior to, passive instruction, especially in terms of knowledge retention and engagement [3], [4], [5].

INSA Toulouse has used a business simulation called SIMGEST for several years in its third-year management course. The pedagogical loop it offers is conceptually simple: each team submits decisions about production, finance and sales, the simulator computes the resulting state of the company, and the teacher uses the published reports to support a debrief. While the underlying idea is sound, the tool itself has aged poorly. Access is restricted, onboarding is not smooth, and the platform is not designed for the larger cohorts and web-first usage habits that characterise current INSA classes. Because the software is proprietary, the school cannot modify it to align with its evolving pedagogical needs or its institutional security and hosting constraints. The result is a teaching activity that still works but is increasingly fragile.

The objective of this project, conducted as part of the fourth-year *Projet d'Initiation à la Recherche* (PIR), is therefore not to compete with SIMGEST on every feature, but to design and implement from scratch a new web-based serious game that (i) preserves the same pedagogical loop: decisions, simulation, results, analysis (ii) is portable to INSA infrastructure, with role-based access for teachers and students, and (iii) is explicitly built as a foundation that successive student teams can extend. Rather than starting with a business scenario, we chose a domain that lent itself to a clearer first iteration: the management of a rugby team competing in a six-round tournament. The strategic decisions available to players: recruiting athletes through an auction and investing in marketing, directly mirrors the kinds of trade-offs encountered in classical business simulations, while the sporting context provides an immediate and engaging narrative.

The rest of this paper is organised as follows. Section II reviews the literature on serious games and their use in education. Section III describes the project methodology. Section IV presents the system architecture and Section V details the database schema. Section VI describes the game mechanics and the underlying formulas. Section VII outlines the student and teacher user interfaces. Section VIII discusses initial results and limitations, and Section IX concludes by outlining the work that future student teams are expected to take over.

II. Literature Review

A. Definition and historical positioning of serious games

The term "serious game" became widespread after the publication of D. Michael and S. Chen's reference work on games intended to educate, train and inform [1]. In the technical report that consolidated the field shortly after, Susi, Johannesson and Backlund defined serious games as digital games used for purposes other than mere entertainment, encompassing military, governmental, corporate, healthcare, political and educational markets [2]. Importantly, serious games are not synonymous with "edutainment" or with simulation: their distinctive feature is that they retain genuine game design qualities: goals, rules, feedback, challenge, while pursuing an additional objective beyond fun [1], [6]. Deterding et al. usefully distinguished serious games, where the

artefact is a full game, from "gamification", defined as the use of game design elements in non-game contexts [6]; our project falls squarely on the serious-game side of this distinction.

B. Learning effectiveness

A growing body of empirical evidence supports the use of serious games as effective learning tools. In a systematic literature review of 129 studies on computer games and serious games for users aged fourteen and above, Connolly et al. reported positive impacts on knowledge acquisition, skill enhancement, motivation and engagement, across a wide range of domains [3]. Wouters et al. later consolidated these findings through a meta-analysis covering 77 effect sizes from 5,547 participants, concluding that serious games are significantly more effective than conventional instruction in terms of learning ($d = 0.29$) and retention ($d = 0.36$), even though their motivational advantage over traditional methods was not statistically robust [4]. Boyle et al. extended this review to studies published between 2009 and 2014 and confirmed the trend [5]. These results are particularly encouraging for active and decision-oriented disciplines such as management.

C. Application domains

Serious games are now applied across multiple domains, each with its own learning objectives and design constraints. In cultural heritage education, Mortara et al. surveyed the use of immersive virtual environments to teach history, art and intangible culture, highlighting both the engagement benefits and the difficulty of preserving historical authenticity [7]. In healthcare, serious games support skill acquisition and clinical decision-making, while in school education they range from mathematics and language learning at the elementary level to higher-education applications [3], [5]. Of particular relevance to the present work, Faisal et al. carried out a recent systematic review of business simulation games in higher education and found consistent evidence that such games improve knowledge acquisition, decision-making skills and learner engagement compared to traditional teaching methods [8]. This is precisely the pedagogical niche that SIMGEST currently occupies at INSA, and that the present project aims to refresh with a modern, web-based alternative.

D. Technological approaches and adaptation

On the technical side, serious-game research has increasingly drawn on artificial intelligence and immersive technologies to improve the player experience. Dynamic difficulty adjustment (DDA), in particular, has been studied as a way to keep learners in the "flow" zone where the challenge matches their skill. Seyderhelm and Blackmore reviewed 59 papers on DDA in serious games and reported a predominance of rule-based and physiological approaches, with a recent shift toward machine-learning methods [9]. While DDA is well documented for single-player games, its translation to multiplayer settings remains challenging because the difficulty experienced by one team is partly produced by the decisions of the others. Konert et al. specifically examined collaborative learning and "game mastering" in multiplayer serious games and emphasised the central role of an instructor able to observe, parametrise and steer the game during runtime [10].

This view directly informs our decision to give the teacher a dedicated control panel with the ability to launch, pause and close rounds, and to expose a small set of global parameters affecting all teams.

E. Design and methodological challenges

Despite these promising results, the literature consistently identifies a number of recurrent design challenges. A central one is the tension between pedagogical rigour and ludic engagement: a game that is too didactic loses its motivational pull, while a game that is too entertaining may fail to cover its learning objectives [11]. Dimitriadou et al. interviewed educators involved in serious-game projects and reported that the most cited challenge was the difficulty of reaching agreement between game designers and domain experts, particularly when institutional rules constrain the freedom of the design [11]. A second recurring issue is methodological: the diversity of evaluation frameworks used in the field makes it hard to compare studies and to consolidate evidence [3], [5]. Finally, encoding the controlled randomness of the real world in a way that feels fair and reproducible without making the game either deterministic or chaotic is a non-trivial design problem, especially in multiplayer contexts where fairness across teams is essential [9], [10].

F. Research gap and positioning of this work

In short, the literature confirms that serious games are an appropriate tool for management education and that their effectiveness is now reasonably well established [3], [4], [5], [8]. Known open problems include the lack of standardised evaluation frameworks, the difficulty of striking a balance between fun and learning [11], and the comparative immaturity of adaptive and multiplayer designs [9], [10]. The present work does not aim to solve these. Its contribution is more modest but immediately useful: to provide INSA with a domain-specific, institutionally hosted, web-based serious game that revives the pedagogical loop currently supported by SIMGEST and that is explicitly designed to be extended over time.

III. Methodology and Project Management

Given the institutional constraints surrounding the project: a small five-student team, a single semester, uncertainty about parts of the hosting environment, and a deliverable explicitly meant to be reused by future cohorts, we adopted an agile, sprint-based methodology inspired by the Scrum framework. Three properties of agile development matched our context particularly well. First, key constraints (notably the integration with INSA's authentication and hosting infrastructure) were not fully known at the start of the project; agile development allowed us to absorb this uncertainty instead of trying to pin everything down up front. Second, our supervisors expected to see a working pedagogical loop early: the principle of an "evolutionary" delivery, where each iteration produces a demonstrable slice of decisions–simulation–results, fits this expectation directly. Third, because the deliverable is a foundation for future teams, freezing each sprint into a cumulative, documented increment was more valuable than producing a throwaway prototype.

Tasks were managed through a Jira backlog organised in eight epics covering authentication and role-based access, game-instance lifecycle, team management, round orchestration, the simulation engine, the user interfaces for both roles, infrastructure and deployment, and documentation. The work was then divided into seven two-week sprints scheduled from early February to mid-May. A typical sprint cycle consisted of a kick-off meeting to refine the backlog, distributed development with twice-weekly informal check-ins, and a sprint-end review with one of the supervisors. Configuration management relied on a single Git repository, with a main branch protected by code review and short-lived feature branches for each user story. Because tests can deter rather than encourage contributions from less-experienced developers, we deliberately restricted the formal testing requirement to the simulation engine and the round-orchestration state machine, where fairness bugs would have the most damaging educational consequences; the remainder of the codebase is validated through review and manual end-to-end tests. The agile cadence proved essential when the team had to absorb several sources of instability such as the absence of a prior research semester to define the subject, the initially non-technical nature of the assigned project, a change in client expectations and supervision after the semester had already begun, the departure of a team member partway through the semester, and more standard scheduling disruptions related to exams and assignments.

IV. System Architecture

A. Stack and decoupled architecture

The system is implemented as two independent applications communicating through a REST interface, a structure deliberately chosen to maximise pedagogical clarity for future student teams and to separate concerns cleanly. The backend is a Node.js server written in TypeScript and built on the Express framework. It exposes the API under the `/api/games` path and never serves HTML directly. Persistence is handled through the Prisma ORM, which generates a typed client from a single `schema.prisma` file and applies migrations to the underlying relational database. The development database is a SQLite file (`dev.db`), with a clean migration path toward the PostgreSQL instance that INSA's CSN service provides for production. The frontend is a React 19 single-page application bundled with Vite and styled with Tailwind CSS v4, using `react-router-dom` v7 for client-side routing. Vite is configured to proxy all `/api` requests to the backend during development, so that the frontend and backend can be developed and deployed independently while behaving as a single application from the user's point of view.

B. MVC organisation of the codebase

The backend follows a classical Model-View-Controller pattern adapted to a REST setting where the "view" is a JSON document rather than an HTML page. The Model layer is defined entirely in `prisma/schema.prisma` and exposed to the rest of the application through a single, lazily-instantiated Prisma client (`src/lib/prisma.ts`). The Controller layer lives in `src/routes/games.ts`, which exposes the fourteen REST endpoints that drive the game: listing and

creating game instances, joining a game by code, fetching the full state of a game, assigning students to teams, updating the game parameters, submitting bids, marketing investments and lineups, signalling team readiness, and triggering round transitions. Each endpoint validates its inputs, enforces business rules (for instance, that a team cannot exceed its budget when placing a bid), invokes the appropriate helper functions, and finally returns a JSON document. The "View" is the React frontend, organised into four pages: a home page, a teacher dashboard, a per-game teacher management page, and a guarded student page that handles join, waiting and gameplay screens.

This organisation preserves a strict boundary between platform concerns and game concerns. The authentication, role inference, game-instance ownership, team membership and round-state machinery are all platform-level functions. The rugby-specific rules such as positions, player generation, the auction mechanism, the scoring formula and the fan dynamics, are localised in a set of helper functions inside the games router and in a small number of constants. A future team wishing to replace the rugby scenario with, for instance, a logistics simulation would only need to rewrite these helpers, not the platform itself.

C. Hosting and deployment

The production environment is provided by INSA's CSN (Centre des Services Numériques), which allocated a virtual server with a PostgreSQL instance, and which constrains access to the platform in two ways: it must be reachable only from inside the INSA network or through the school VPN, and authentication must rely on the institutional CAS module. To keep the platform deployable by future teams without re-discovering the entire stack, the goal is to have an application that is shippable as a small set of Docker images orchestrated by a docker-compose configuration, so that a future cohort can bring the system back online with a single command. Our version includes a "demo authentication" mode where the user explicitly selects a role; the rest of the application is fully ready for CAS integration once the institutional parameters are confirmed.

V. Database Architecture

The persistence layer is the backbone of the platform: it is what makes games saveable and resumable across weeks, it is what guarantees that every state transition leaves a trace, and it is the part that future teams are most likely to extend. We therefore put considerable effort into defining a schema that is small enough to be understood at a glance, but rich enough to support the full six-round loop without ad-hoc additions.

A. Overview of the entities

The schema is composed of eleven entities, organised around three concentric layers. The outermost layer captures the institutional context: a User has a name and a role (TEACHER or STUDENT), and a GameInstance has a name, a unique join code, a status (CREATED, ACTIVE, PAUSED or FINISHED), a current round number and an owning teacher. The middle layer captures the in-game state: a Team belongs to exactly one GameInstance and holds the time-varying values

that drive the game: available budget, current fan count, advertising score (pubScore), merchandising score (merchScore), accumulated points, and the win/draw/loss counters used to build the league table. The innermost layer captures the per-round events: Bid, MarketingDecision and MatchResult.

B. Membership and access

Two junction tables, GameParticipant and TeamMember, mediate the relationship between users and games. GameParticipant records every student who has entered a game through its join code, whether or not they have been assigned to a team yet. This intermediate state matters: the platform must be able to tell the teacher who has logged in but is still waiting for an assignment, which is precisely the use case that drives the drag-and-drop view described in Section VII. TeamMember then records the actual assignment of a student to one of the four teams of a game, with a uniqueness constraint preventing a student from belonging to two teams within the same game.

C. Players, market and decisions

Players are first-class entities of the schema, not embedded in JSON blobs. A Player has a name, a position, a numeric skill (stats), a price, and two boolean flags indicating whether it is currently on the market and whether it has been sold. Each Player belongs to a single GameInstance, which means that two games are entirely isolated even if they are played in the same class. A TeamPlayer junction associates Players with Teams and remembers, for each association, whether the player is in the starting lineup. Bids and MarketingDecisions are stored per round, with a uniqueness constraint guaranteeing that a team can only submit one bid for a given player in a given round and one marketing decision per round. The MatchResult table closes the loop: for each simulated match it records the two teams, the two scores, the outcome on each side, and the resulting deltas in fans and money. This per-round history is what makes the report tab (described in Section VII) possible: it is not a recomputation but a replay of stored data, which is also the property that allows games to be paused and resumed without ambiguity.

D. Extensibility

Two design choices make the schema easy to extend. First, the global parameters of a game are isolated in a dedicated GameParameter entity with a one-to-one relationship to GameInstance. Adding a new global variable like a sponsorship multiplier or a stadium-capacity factor is therefore a localised change that does not affect other entities. Second, the schema deliberately avoids embedding game-specific logic in column names: a column called "stats" rather than "tackleScore", or "pubScore" rather than "billboardCount", will keep its meaning even if the rugby theme is replaced. As a result, future teams can layer new mechanics on top of the existing schema with very few destructive migrations.

VI. Game Design and Mechanics

A. Core loop and round structure

A complete game is composed of exactly six rounds, played by four teams that start with identical initial conditions (50,000 in starting budget, 1,000 fans, identical eight-player rosters with initial skill 40 each, advertising and merchandising scores at zero). Each round follows the same three-phase structure. During the decision phase, every team chooses which players to start, which players on the market to bid on, and how much to invest in advertising and merchandising. During the simulation phase, the engine resolves the auctions, applies the marketing decisions, computes the match results based on the resulting team strengths, updates the fans and the budget, and writes a MatchResult row. During the results phase, players observe the new league table and review the impact of their decisions through a dedicated report screen. The teacher then opens the next round when ready.

B. Market and auction

At the start of each round, fifteen new players are generated and exposed on the market. To preserve gameplay balance over the six rounds, both the player statistics and the player prices scale with the round number. The skill of a generated player follows

$$\text{stats} \in [40 ; 40 + (\text{round} \times \text{rand}(1, 10))]$$

so that later rounds expose progressively stronger talent without making the early market obsolete. The starting price of a player is a quadratic function of skill plus a linear function of round number,

$$\text{price} = \text{stats}^2 / 10 + \text{round} \times 100$$

which ensures that top-tier players become substantially more expensive than the squared scaling alone would predict, preventing dominant teams from accumulating elite rosters too quickly. Auctions are sealed-bid: every team submits hidden bids during the decision phase, and the highest bid at the close of the round wins the player. Ties are broken by submission time, and a team can never bid more than its current budget.

C. Marketing investments

Each team can invest in two marketing tracks, an advertising score (pubScore) and a merchandising score (merchScore), both capped at ten points and limited to two investments per round per track. The cost of an investment grows with the round number and with the current score in the chosen track,

$$\text{cost} = \text{round} \times 7000 \times (1 + \text{current_score} / 10)$$

so that early investments are cheap but the marginal cost grows steadily, discouraging overinvestment strategies during the first rounds.

D. Match outcome

The performance of a team in a match is a weighted multi-factor score combining the average skill of its starting lineup, its current fan base, its combined marketing scores, and a per-round luck component:

$$\begin{aligned} \text{team_score} = & 0.5 \times (\text{avg_stats} / 100) \\ & + 0.2 \times (\text{fans} / \text{MAX_FANS}) \\ & + 0.1 \times ((\text{pubScore} + \text{merchScore}) / 20) \\ & + 0.2 \times \text{luck} \end{aligned}$$

The luck term is deterministic per (team, round) pair, a seeded value derived from the team identifier and the round number, so that two simulations of the same round on the same team produce identical outcomes. This makes the engine fully reproducible, which is essential both for debugging and for defending the perceived fairness of the game to students. The two teams of a match are paired according to a round-robin schedule that guarantees that each team plays every other team exactly twice over the six rounds. The team with the higher final score wins (three points); ties are awarded one point each, and losses score zero.

E. Fan and financial dynamics

After each match, the fan count of each team evolves according to a controlled, non-exponential rule. A multiplicative gain proportional to the marketing investment is added, complemented by a result-dependent bonus (+5,000 for a win, +2,000 for a draw, -1,000 for a loss), with the total fan count clamped between zero and one million. The team's revenue for the round is then proportional to both the round number and the current fan base, with a winning team receiving an additional sporting bonus proportional to the round number. The combination of these rules creates the feedback loop that the project deliberately aimed to produce: sporting results increase fans, fans increase revenue, and revenue funds better players for the following rounds.

VII. User Interfaces

The platform exposes two role-specific interfaces, deliberately distinguished both visually and in terms of available actions. The high-level use cases of each role are summarised in Figs. 1 to 3.

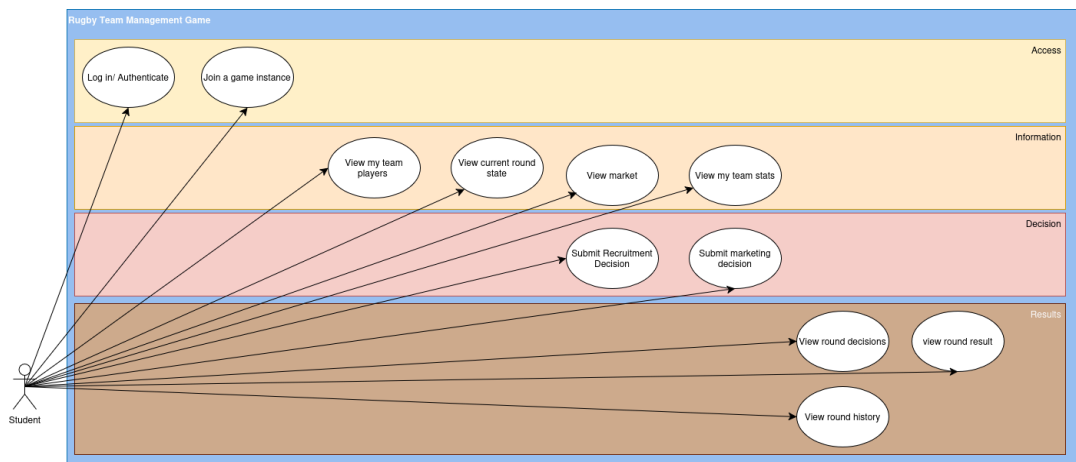


Fig. 1. Student-side use case iagram, grouped by access, information, decision and results.

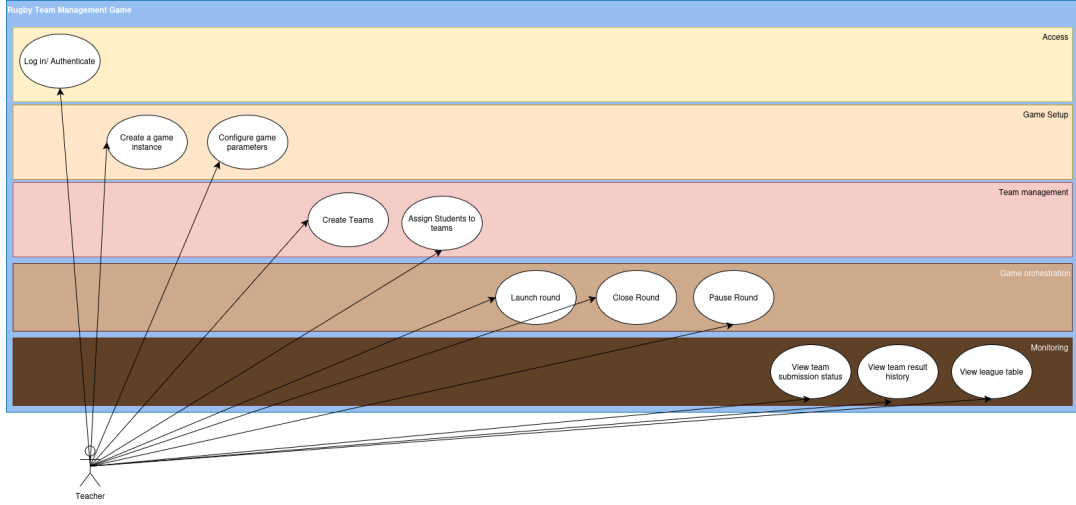


Fig. 2. Teacher-side use case diagram, covering access, game setup, team management, round orchestration and monitoring.

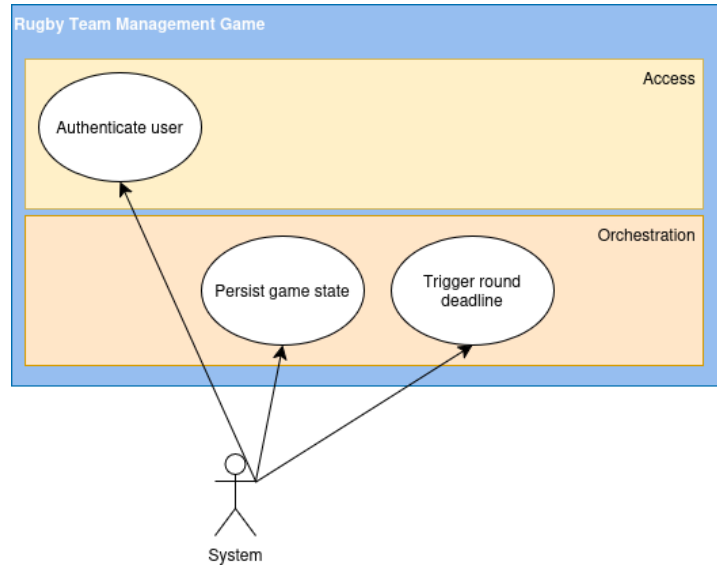


Fig. 3. System-level use cases, capturing user authentication, game-state persistence and round-deadline triggers.

A. Student-side experience

Authentication is intended to be delegated to the CSN identity provider through a CAS module; in the demonstration mode used for V1, the student selects the student role explicitly. The first screen thereafter is the join screen, where the student enters the unique code provided by the teacher for their game instance. If the code is valid, the student is registered as a GameParticipant of that instance and shown a waiting screen until the teacher assigns them to one of the four teams. From that point on, the interface adopts a stable structure: a fixed header displaying the team name, the current fan count and the available budget, and a tab bar with five sections: Team, Market,

Marketing, Leaderboard and Report. The Team tab is the squad management screen, where players can reorder the starting lineup; the Market tab implements the sealed-bid auction described in Section VI; the Marketing tab exposes the advertising and merchandising investments; the Leaderboard tab shows the league table and upcoming fixtures (Fig. 4); and the Report tab provides a historical breakdown of past rounds (Fig. 5).

Leaderboard								
Round 0								
PLACE	TEAM	M	W	D	L	PD	PTS	FORM
1	Team 1	0	0	0	0	0	0	● ● ● ● ●
2	Team 2	0	0	0	0	0	0	● ● ● ● ●
3	Team 3	0	0	0	0	0	0	● ● ● ● ●
4	Team 4	0	0	0	0	0	0	● ● ● ● ●

Fig. 4. Student leaderboard screen: standings table showing matches played, wins, draws, losses, point differential, total points and recent form for the four teams.

Game 1

Team 1

Locked

FANS1,000

BUDGET€50,000

Leave game

Team

Market

Marketing

Leaderboard

Report

Report

Historical performance snapshot.

PLACE	TEAM	FANS	MONEY	RESULT	GROWTH	EVENTS	ROUND
1	Team 1	1,000	€50,000	Pending	0 fans	0	0
2	Team 2	1,000	€50,000	Pending	0 fans	0	0
3	Team 3	1,000	€50,000	Pending	0 fans	0	0
4	Team 4	1,000	€50,000	Pending	0 fans	0	0

Fig. 5. Student report screen: historical performance snapshot showing each team's place, fans, money, result, growth, events and round number.

B. Teacher-side experience

The teacher dashboard is conceived as a control hub rather than a game interface. After authentication, the teacher lands on a list of their previously created games, each shown with its name and join code, and can create a new game with a single button. Selecting a game opens a dedicated management page with five tabs: Teams, Parameters, Round Management, Report and Leaderboard. The Teams tab allows the teacher to assign students to teams using a drag-and-drop mechanism between a list of unassigned students and the four team containers (Fig. 6). The Parameters tab exposes the four global variables (luck factor, fan growth, financial growth and injury probability) that the teacher can adjust between rounds to influence difficulty. The Round Management tab is the operational control centre, providing explicit Launch, Stop and Move to next

round controls so that the teacher decides when decisions can be made, when they are finalised and when the game advances; the screen also displays the current round, the game state and how many teams have signalled that they are ready (Fig. 7). The Report and Leaderboard tabs mirror their student counterparts, except that the teacher has visibility on all four teams simultaneously, which supports class-wide debriefs.

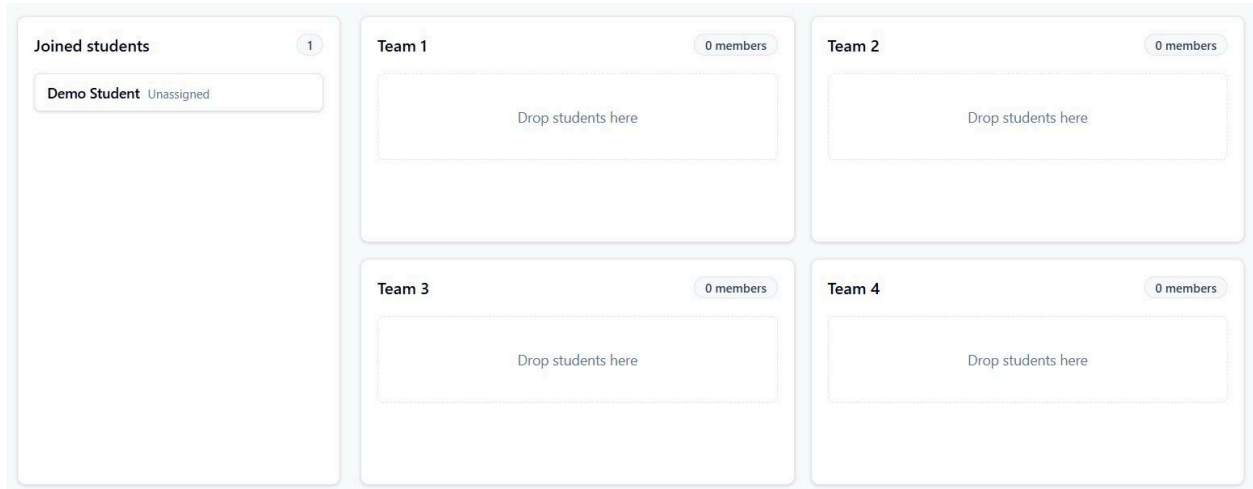


Fig. 6. Teacher team-assignment screen: drag-and-drop interface showing the list of joined students and the four team containers used for student assignment.

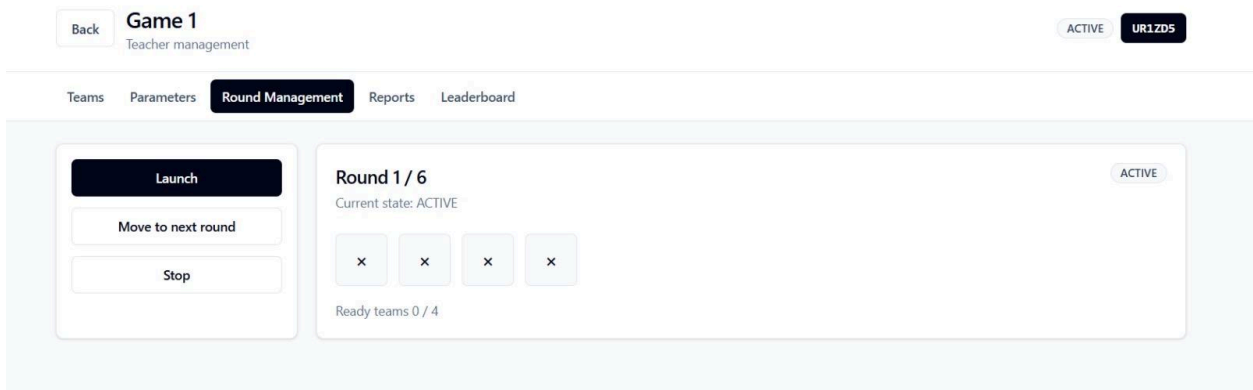


Fig. 7. Teacher round-management screen: controls for launching, stopping and advancing the game round, together with the current round state and team-readiness indicator.

VIII. Results and Discussion

At the time of writing, the platform implements the complete pedagogical loop end-to-end. A teacher can create a game, share its join code, observe students entering the waiting screen, assign them to teams through drag-and-drop, launch the first round, watch the team-readiness indicator update as each team marks itself ready, close the round, trigger the simulation, observe the resulting league table, advance to the next round and repeat this six times. Across our internal play-tests, this loop has executed reliably for four parallel teams without inconsistencies in the

database, and the history exposed in the Report tab matches the sequence of decisions submitted in real time.

The deterministic luck term has proved particularly valuable. Because it is seeded by the team identifier and the round number, two simulations of a round on the same set of inputs produce identical match results; this not only simplifies debugging during development, but also gives the teacher a clear and defensible answer when students ask why a particular result occurred. The controlled fan-evolution rule, capped at one million and complemented by result-dependent bonuses, prevents the runaway dynamics that early prototypes of the engine exhibited, where a team that won its first match could rapidly become unbeatable. Across simulated six-round tournaments, the final league position is sensitive both to early decisions (notably the first auction) and to mid-round marketing investments, which is the qualitative pattern the design aimed to produce.

Several limitations should be acknowledged. First, the V1 authentication is in a demonstration mode rather than on the institutional CAS module; the integration is ready in the application code but requires final configuration from the CSN service. Second, the test coverage is intentionally limited to the simulation engine and round orchestration, which means that future regressions in less critical parts of the application may go unnoticed until they manifest in production. Third, the player market is still very simple: players have a single global skill rather than positional attributes, and the auction is sealed-bid rather than live. These choices were deliberate as they were necessary to deliver a complete loop within a single semester but they are also the most obvious starting points for future development.

IX. Conclusion and Future Work

This work has presented the design and first implementation of a web-based serious game intended to replace the ageing SIMGEST simulation used in INSA's third-year business course. The game is structured around the management of a rugby team competing in a six-round tournament, with strategic decisions involving recruitment, marketing and squad management. The implementation rests on a TypeScript/Express REST API backed by an eleven-entity relational schema managed through the Prisma ORM, paired with a React/Tailwind frontend. The platform is designed to be hosted on INSA infrastructure, with role-based access for teachers and students, and is explicitly built as a foundation that future student teams can extend. A complete pedagogical loop, from login through team assignment and round simulation to results publication, has been demonstrated.

Several extensions are natural candidates for future cohorts. The most pressing is the finalisation of CAS authentication so that the demonstration login can be replaced with the institutional identity flow. Beyond authentication, the player market can be enriched with positional attributes, multiple skills and live auctions; risk events such as injuries and scandals, currently sketched in the schema, can be developed into a more substantial sub-system; and the match resolution itself could be augmented with a short, visual real-time animation rather than a pure formula evaluation,

reinforcing the entertainment dimension of the game. More ambitiously, because the platform engine is deliberately separated from the rugby-specific rules, future teams can also reuse the infrastructure to instantiate entirely new scenarios, for instance, replacing rugby with a classical business simulation, and bringing the project full circle back to its original SIMGEST inspiration.

Acknowledgements

The authors would like to thank Oumaima El Haddadi for her continuous guidance and patience throughout the project. As both client and technical supervisor, she ensured frequent meetings, provided detailed advice on the code architecture and the structure of the present report, and acted as the communication link with the CSN service for all hosting and authentication questions. We are equally grateful to Paul Scanlan, the main client of the project alongside Oumaima, for clarifying the project expectations, the pedagogical use case and the long-term vision of the platform, and for providing constructive feedback at each sprint review. We also thank the CSN (Centre des Services Numériques) of INSA Toulouse for providing the hosting infrastructure and for their technical advice on deployment, database provisioning and authentication.

Declaration on the use of AI

In accordance with the evaluation criteria of the PIR programme, the authors disclose that an Anthropic's Claude was used during the preparation of this report for review and grading of the IEEE format, the english grammar and other syntax and lexical errors that might have slipped in.

Open AI's ChatGPT was used for the following formal-support tasks: (i) helping to structure the article and to map our existing notes onto an IEEE-style article layout; (ii) locating candidate bibliographic sources and formatting them in IEEE convention; (iii) suggesting wording improvements and proofreading the manuscript for grammar, style, and consistency; (iv) drafting technical descriptions of the system from the source code and database schema that we developed, which we then reviewed, corrected, and adapted to reflect our own understanding and design decisions; (v) summarizing project meetings and drafting follow-up emails for team members or stakeholders who could not attend; (vi) organizing requirements collected from the different clients and supervisors, including identifying possible points of conflict between expectations; (vii) supplementing the team's own feature proposals by suggesting additional functionality that could plausibly support the client's pedagogical goals; and (viii) assisting with debugging by analyzing error messages, logs, and relevant code excerpts when a bug had already resisted several hours of team investigation.

All design decisions, final code, database schema, game formulas, supervisor meetings, sprint planning, deployment work and play-tests were carried out by the very human authors. After the AI-assisted drafting, the manuscript was reviewed, edited and validated by the authors and their supervisors, and the corrections and modifications applied to the final text reflect their original work and their interpretation of the project. The authors take full responsibility for the content of this report.

References

- [1] D. R. Michael and S. Chen, *Serious Games: Games That Educate, Train, and Inform*. Boston, MA, USA: Thomson Course Technology, 2005.
- [2] T. Susi, M. Johannesson, and P. Backlund, "Serious Games – An Overview," School of Humanities and Informatics, University of Skövde, Sweden, Tech. Rep. HS-IKI-TR-07-001, Feb. 2007.
- [3] T. M. Connolly, E. A. Boyle, E. MacArthur, T. Hainey, and J. M. Boyle, "A systematic literature review of empirical evidence on computer games and serious games," *Computers & Education*, vol. 59, no. 2, pp. 661–686, Sep. 2012, doi: 10.1016/j.compedu.2012.03.004.
- [4] P. Wouters, C. van Nimwegen, H. van Oostendorp, and E. D. van der Spek, "A meta-analysis of the cognitive and motivational effects of serious games," *Journal of Educational Psychology*, vol. 105, no. 2, pp. 249–265, May 2013, doi: 10.1037/a0031311.
- [5] E. A. Boyle, T. Hainey, T. M. Connolly, G. Gray, J. Earp, M. Ott, T. Lim, M. Ninaus, C. Ribeiro, and J. Pereira, "An update to the systematic literature review of empirical evidence of the impacts and outcomes of computer games and serious games," *Computers & Education*, vol. 94, pp. 178–192, Mar. 2016, doi: 10.1016/j.compedu.2015.11.003.
- [6] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining “gamification”," in *Proc. 15th Int. Academic MindTrek Conf.: Envisioning Future Media Environments*, Tampere, Finland, Sep. 2011, pp. 9–15, doi: 10.1145/2181037.2181040.
- [7] M. Mortara, C. E. Catalano, F. Bellotti, G. Fiucci, M. Houry-Panchetti, and P. Petridis, "Learning cultural heritage by serious games," *Journal of Cultural Heritage*, vol. 15, no. 3, pp. 318–325, May 2014, doi: 10.1016/j.culher.2013.04.004.
- [8] M. M. Faisal, M. Chaudhry, S. Saleem, and N. Mian, "Business simulation games in higher education: A systematic review of empirical research," *Human Behavior and Emerging Technologies*, vol. 2022, art. no. 1578791, 2022, doi: 10.1155/2022/1578791.
- [9] A. J. A. Seyderhelm and K. L. Blackmore, "Systematic review of dynamic difficulty adaption for serious games: The importance of diverse approaches," *Simulation & Gaming*, vol. 52, no. 6, pp. 758–780, Dec. 2021, doi: 10.1177/10468781211050810.
- [10] J. Konert, S. Göbel, and R. Steinmetz, "Collaborative learning and game mastering in multiplayer games," in *Serious Games and Virtual Worlds in Education, Professional Development, and Healthcare*, K. Bredl and W. Bösch, Eds. Hershey, PA, USA: IGI Global, 2013, pp. 85–104, doi: 10.4018/978-1-4666-3673-6.ch006.
- [11] A. Dimitriadou, N. Djafarova, O. Turetken, M. Verkuyl, and A. Ferworn, "Challenges in serious game design and development: Educators’ experiences," *Simulation & Gaming*, vol. 52, no. 2, pp. 132–152, Apr. 2021, doi: 10.1177/1046878120944197.